

Grounded Cybersecurity RAG for Auditable Infrastructure Risk Assessment and Security Report Generation

Dr S Ramraj
Associate Professor

Department of Computer Science and Engineering
Rajalakshmi Engineering College, India

Sachin.N

Department of Computer Science and Engineering
Rajalakshmi Engineering College, India

Sarvesh.R

Department of Computer Science and Engineering
Rajalakshmi Engineering College, India

Sarvesh Ravi

Department of Computer Science and Engineering
Rajalakshmi Engineering College, India

Sivaruhith.S

Department of Computer Science and Engineering
Rajalakshmi Engineering College, India

Abstract

Infrastructure-security assessment commonly combines network discovery, vulnerability knowledge, configuration guidance, and human-readable reporting. Large language models can help synthesize such evidence, but security reporting introduces a strict requirement for traceable claims: a fluent explanation is not sufficient when a cited CVE, severity statement, or remediation recommendation cannot be verified. This paper presents a grounded retrieval-augmented generation framework for cybersecurity health assessment from Nmap network-scan data. The framework parses Nmap XML, applies deterministic rules for security/risk analysis, retrieves vulnerability and hardening evidence from CVE-oriented and CIS knowledge sources, constructs a grounded context, and uses a Gemini-based language model for report synthesis. Citation validation and explicit UNSCORED or abstention states constrain unsupported security claims. The implementation includes a Flask API, Next.js interface, report export, audit logging, automated browser-level testing, and quota-aware degradation behavior. Evaluation is deliberately separated into retrieval and generation-related measures. On a 1,325-CVE evaluation derived from the Vul-RAG benchmark, 80% of entries form a temporary retrieval knowledge base and 20% are held out. Paraphrased knowledge queries achieve 51.6% Top-1 and 72.4% Top-5 retrieval, while a similarity threshold of 0.4 increases precision to 91.6% with coverage reduced to about 18%. In a separate grounded-versus-ungrounded citation experiment, 21 of 58 grounded cases issue citations and all 21 are correct, whereas the ungrounded condition issues 0 of 58 citations. These findings

characterize retrieval and citation behavior and motivate confidence-aware abstention as future work.

Index Terms—retrieval-augmented generation, cybersecurity, infrastructure security, Nmap, CVE, CIS Benchmarks, grounded generation, citation verification, hallucination mitigation.

I. INTRODUCTION

A. Cybersecurity Context

Infrastructure-security assessment must translate heterogeneous technical observations into decisions that can be reviewed, prioritized, and acted upon. Network scanners expose hosts, ports, services, versions, and other observations, but raw scan output is not itself a security assessment. A useful assessment must connect observed exposure to security significance, relevant vulnerability knowledge, hardening guidance, and remediation-oriented explanation. Nmap provides XML output intended for programmatic processing, making it suitable as a structured input to automated analysis pipelines [1].

The proposed framework addresses this reporting problem by treating network evidence and security knowledge as distinct stages. Nmap observations first pass through deterministic analysis rather than being interpreted directly by a language model. The resulting findings become retrieval queries against vulnerability and security-guidance sources. Retrieved evidence is then supplied to a Gemini-based synthesizer that generates an audience-aware report. This ordering separates risk classification from natural-language generation and makes the narrative layer dependent on explicit evidence.

B. Limitations of LLM-Based Security Reporting

Large language models can produce coherent explanations while still generating unsupported or factually incorrect statements. Research on hallucination distinguishes factuality problems from failures of faithfulness to supplied context, and surveys identify hallucination as a continuing reliability concern for deployed language-generation systems [2], [3]. In cybersecurity, the consequences are especially direct. An invented CVE identifier, an incorrect affected version, or an unsupported remediation step can misdirect triage and weaken the auditability of a report.

Retrieval-augmented generation reduces reliance on model-internal knowledge by conditioning generation on external evidence [4]. However, retrieval alone does not guarantee correctness. A retriever may return a semantically similar but incorrect vulnerability, while a generator

may ignore or overextend the retrieved context. A security-reporting pipeline therefore requires additional controls for citation eligibility, evidence sufficiency, and abstention.

C. Research Gap

Knowledge-grounded vulnerability research has recently demonstrated the value of retrieval for source-code vulnerability detection. Vul-RAG constructs a vulnerability knowledge base from historical vulnerability and fix information and uses retrieved knowledge to support vulnerability reasoning over code [5]. The task addressed by the present framework is different. Its input is Nmap network-scan information rather than source-code functions, and its output is an auditable infrastructure-security report rather than a vulnerable-versus-patched classification.

The resulting gap concerns the application of grounded security reasoning to infrastructure observations. The system must combine deterministic findings, vulnerability evidence, configuration guidance, citation validation, and report generation while retaining the ability to abstain when evidence is insufficient. The contribution is therefore best characterized as an infrastructure-security application of grounded RAG rather than a reproduction of source-code vulnerability detection.

D. Proposed Approach

The framework follows a deterministic-first pipeline: Nmap XML is parsed into structured observations; rule-based analysis assigns security-related properties and prioritization; CVE and CIS retrieval supplies evidence; a grounded context is assembled; Gemini generates the narrative; and a validation layer checks citations before the report is finalized. When evidence does not satisfy the system's requirements, the workflow can represent the result as UNSCORED, no-grounded-citation, or manual-review-required rather than forcing a fabricated answer.

The implementation also exposes the workflow through a Flask API and Next.js interface and supports report-oriented outputs including PDF/JSON export and audit logging. Quota-aware degradation and a MockClient fallback are documented as operational safeguards rather than as changes to the security decision logic.

E. Research Objectives

- Determine whether lexical retrieval can identify relevant CVE knowledge from truncated and paraphrased queries.
- Measure how a similarity threshold changes retrieval precision and coverage.

- Characterize citation issuance and correctness under grounded and ungrounded generation conditions.
- Provide an auditable separation between deterministic security analysis and probabilistic language generation.
- Support safe handling of insufficient evidence through explicit abstention or UNSCORED states.

F. Research Questions

- RQ1: How effectively does the TF-IDF retrieval component recover the ground-truth CVE under truncated and paraphrased query construction?
- RQ2: How does similarity-threshold filtering change precision, coverage, and held-out-CVE abstention behavior?
- RQ3: How does evidence grounding affect citation issuance and citation correctness in the evaluated report-generation sample?

The evaluation scope is defined in Section III.F; source-code vulnerability detection and end-to-end network vulnerability discovery are outside the evaluated scope.

G. Contributions

The principal contributions are as follows. First, the framework places deterministic security analysis before LLM synthesis, so severity and finding prioritization do not depend on generated prose. Second, it connects vulnerability evidence with CIS hardening guidance to support report-level remediation context. Third, citation validation restricts the report's security references to evidence that can be checked against the system's knowledge sources. Fourth, explicit UNSCORED and abstention states provide a mechanism for declining unsupported synthesis. Fifth, the evaluation treats retrieval quality and citation behavior as separate scientific objects and documents a measurable precision-coverage trade-off. Finally, the work demonstrates an infrastructure-security use of grounded RAG that is distinct from source-code vulnerability classification.

II. RELATED WORK

A. Automated Vulnerability and Security Assessment

Configuration security is complementary to vulnerability identification. CIS Benchmarks provide consensus-based secure-configuration guidance across operating systems, network devices, cloud platforms, databases, and other technology families [8]. The proposed framework therefore treats vulnerability evidence and hardening evidence as related but distinct knowledge types.

B. Large Language Models in Cybersecurity

LLMs can summarize security observations and translate technical evidence into audience-specific

explanations, but their open-ended generation introduces factuality and traceability risks. Hallucination research identifies fabricated or unsupported content as a persistent issue in language-generation systems [2], [3]. For security reporting, this motivates a design in which the LLM is responsible for synthesis and explanation rather than being the sole source of vulnerability facts.

The proposed framework follows this principle by separating deterministic analysis, retrieval, and generation. The language model receives retrieved evidence rather than an unconstrained request to infer vulnerability information from scan text alone. This architecture moves critical factual constraints into observable system stages while retaining the model's generative role.

C. Retrieval-Augmented Generation

RAG combines a generative model with an external non-parametric memory, allowing retrieved information to condition the generated response [4]. The approach is attractive for security applications because vulnerability and configuration knowledge changes independently of the language model. It also creates an opportunity to attach provenance to generated statements.

The present implementation uses lexical TF-IDF retrieval rather than a dense semantic index. TF-IDF represents documents by term-weighted vectors, with inverse-document frequency reducing the influence of terms that occur broadly across the corpus [9]. Under normalized vector representations, cosine similarity can be used to rank query-document matches. This choice is computationally simple and transparent, but it also creates a known sensitivity to vocabulary overlap and paraphrase.

D. Grounded Generation and Hallucination Mitigation

Grounding is useful only when the retrieved context is relevant and the generation layer respects it. Work on hallucination identifies both factuality and context-faithfulness challenges, while RAG-oriented studies show that external retrieval can reduce unsupported generation without making retrieval itself infallible [2], [10]. The proposed system therefore adds citation validation and explicit abstention concepts on top of retrieval.

The grounded-versus-ungrounded experiment measures citation issuance and correctness within the evaluated sample.

E. Relationship to Vul-RAG

Vul-RAG provides the closest conceptual reference because it applies knowledge-level RAG to vulnerability reasoning [5]. Its primary input is source code and its principal task is vulnerability

detection over vulnerable/patched code pairs. The proposed framework instead begins with Nmap scan observations and produces infrastructure-security reporting. The two systems therefore share a knowledge-grounding principle but operate on different inputs, outputs, and evaluation objectives.

TABLE I. DIMENSIONAL COMPARISON WITH VUL-RAG

Dimension	Vul-RAG	Proposed framework
Input	Source-code functions	Nmap/network scan data
Primary task	Source-code vulnerability detection	Infrastructure security assessment and report generation
Knowledge	Vulnerability/fix knowledge	CVE, CISA KEV, and CIS-oriented evidence
Retrieval	Knowledge-level vulnerability retrieval	TF-IDF over supported security text
LLM role	Vulnerability reasoning	Grounded report synthesis
Output	Vulnerability classification and knowledge	Auditable security report
Evaluation	Detection-oriented metrics	Retrieval, citation, grounding, abstention
Abstention	Not the central reported objective	Explicit UNSCORED/manual-review behavior

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. Input and Security Findings

Let S denote an Nmap XML scan. The parser converts S into structured observations containing host- and service-level information supported by the scan representation. A finding f is derived from an observation through deterministic rules. The framework does not assume that every open service is a vulnerability; rather, rules map observable security conditions to documented security properties and prioritization information.

The conceptual pipeline can therefore be represented as $S \rightarrow F \rightarrow E \rightarrow G \rightarrow V \rightarrow R$, where F denotes deterministic findings, E retrieved evidence, G generated narrative, V citation validation, and R the final report or abstention state.

B. Deterministic Risk Analysis

The deterministic rule engine is intentionally placed before the language model. This separation makes the security-related classification inspectable and prevents generated prose from silently changing a rule-derived severity. The rule engine provides the deterministic risk-scoring and finding-prioritization layer.

The rule engine is an explicit decision layer whose behavior is determined by documented rules.

C. Evidence Retrieval and Grounding

For a query q derived from a finding, the retriever ranks knowledge entries d by lexical similarity. Let $v(q)$ and $v(d)$ denote TF-IDF vectors. A cosine similarity score can be represented as:

$$\text{sim}(q,d) = (v(q) \cdot v(d)) / (\|v(q)\| \|v(d)\|) \quad (1)$$

The evaluation uses the similarity score as a basis for ranking and threshold experiments. A threshold τ defines an evidence-acceptance condition: a candidate can be passed to the synthesis stage only when its score satisfies the evaluated threshold rule. The production pipeline's existing citation safeguard is separate: only CVE identifiers present in the knowledge base are eligible for citation. The production safeguard is evaluated separately from the retrieval benchmark.

D. Citation Requirement

A citation is accepted when the generated CVE identifier corresponds to a vulnerability record available in the knowledge base and the retrieved evidence is relevant to the finding. The validation stage checks identifier eligibility and evidence support before a report is marked grounded. If a generated citation cannot be supported by the retrieved evidence, the finding can enter a no-grounded-citation, UNSCORED, or manual-review state. This requirement links each accepted vulnerability reference to an inspectable evidence record and makes citation validation an explicit part of the reporting pipeline.

E. Abstention and Unscored States

The framework treats insufficient evidence as a legitimate output state. UNSCORED can represent a finding for which a defensible synthesis cannot be produced. No-grounded-citation and manual-review-required states provide additional ways to preserve uncertainty instead of converting it into fabricated certainty. The threshold experiment investigates the retrieval side of this design principle by measuring how many held-out CVEs can be rejected when similarity is too low.

An important scope distinction remains: the evaluation's threshold-based abstention behavior is an experimental retriever study and is not identical

to the production citation-eligibility safeguard. the implementation explicitly identifies the absence of a confidence cutoff in the existing similarity search as a future-work item.

F. Scope and Assumptions

The framework is designed for infrastructure-security assessment from Nmap network-scan observations. The evaluation focuses on CVE retrieval, confidence-aware filtering, and citation behavior. The CVE retrieval benchmark contains 1,325 Linux-kernel CVEs derived from the Vul-RAG dataset, whereas the application input is Nmap data; this difference is treated as a deliberate evaluation-scope constraint. The study assumes syntactically valid Nmap observations, availability of the supported security knowledge sources, and deterministic rules that provide the initial security findings. The language model is used for evidence-grounded synthesis after those stages. The reported metrics therefore characterize the evaluated retrieval and citation components under the documented operating conditions.

IV. PROPOSED FRAMEWORK AND SYSTEM ARCHITECTURE

A. System Overview

Figure 1 summarizes the end-to-end architecture. The system begins with Nmap XML, parses the scan into structured findings, applies deterministic security rules, retrieves CVE and CIS evidence, constructs a grounded context, and invokes Gemini for narrative synthesis. Citation validation then determines whether the resulting report can be accepted as grounded. If the evidence state is insufficient, the system can return an UNSCORED or related abstention state.

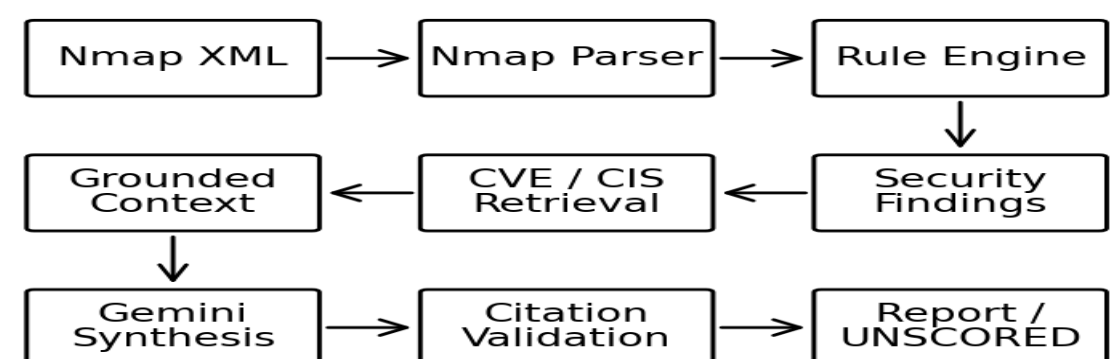


Fig. 1. Overall system architecture for deterministic analysis, evidence retrieval, grounded synthesis, and citation validation.

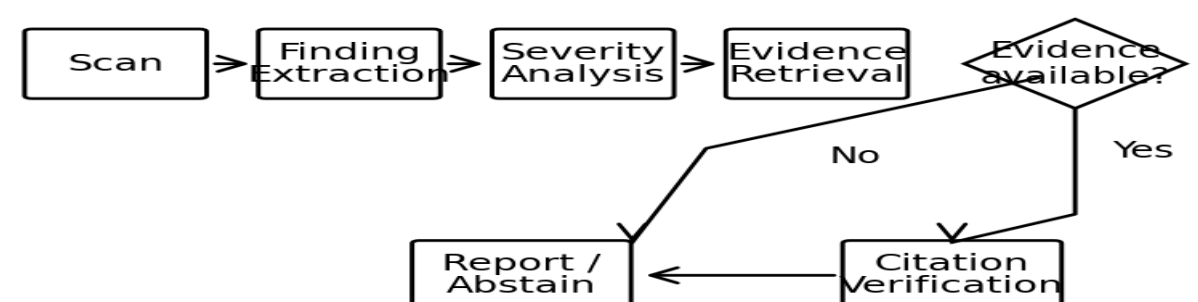


Fig. 2. Security-analysis and grounded-RAG workflow emphasizing the separation of deterministic finding analysis, evidence retrieval, generation, and validation.

The architecture deliberately assigns different responsibilities to deterministic and generative

components. Parsing and rule analysis transform machine-readable observations into explicit security findings. Retrieval supplies external knowledge. Generation produces a human-readable explanation. Validation controls which security references can appear in the accepted report.

B. Nmap Scan Parsing

Nmap XML is used as the ingestion format because it is structured for software processing and contains host, port, service, version, and script-related information [1]. The `nmap_parser.py` component converts the XML representation into the internal finding-oriented representation. Parsing occurs before security reasoning, reducing the risk that a language model interprets raw XML inconsistently.

The parser is an extraction layer, not a vulnerability oracle. Its output is therefore treated as observed scan information. Vulnerability and hardening conclusions are introduced only by the subsequent rule and knowledge layers.

C. Deterministic Security and Risk Analysis

The `rule_engine.py` component applies explicit criteria to parsed findings. Severity scoring and finding prioritization are calculated before LLM synthesis. This deterministic-first design provides a stable control point for auditability: the narrative generator cannot independently redefine the risk score through natural-language reasoning.

The separation is also useful operationally. If an LLM request fails because of quota or service availability, the deterministic finding remains available. The report can therefore degrade to a lower-level evidence state rather than fabricating a complete narrative.

D. CVE and CIS Knowledge Retrieval

The retrieval layer supports vulnerability and configuration knowledge. CVE-oriented evidence provides vulnerability identity and descriptive context, while CIS-oriented evidence supplies secure-configuration and remediation guidance. CISA KEV information is identified in the implementation as an additional knowledge source for vulnerability prioritization. These sources are complementary rather than interchangeable: a CVE record can describe a vulnerability, while a CIS recommendation can describe a configuration state or hardening action.

Eight of eight evaluated CIS mappings matched the expected guidance. This mapping result is

reported separately from CVE retrieval accuracy and citation precision.

E. Retrieval-Augmented Context Construction

`retrieval.py` supplies ranked evidence to `synthesize.py`. The context is constructed from retrieved records rather than from unconstrained model memory. TF-IDF provides the lexical representation used in the documented evaluation. This design favors interpretability and low operational complexity but is vulnerable to terminology mismatch, synonyms, and paraphrase. The evaluation results later demonstrate this limitation.

The retrieval stage is also the natural location for confidence-aware filtering. The evaluation shows that imposing a similarity threshold can sharply increase precision while reducing coverage. Because the threshold is not yet integrated into the production retrieval pipeline, the measured trade-off is presented as evidence for future engineering rather than as an already-deployed feature.

F. Grounded Gemini Synthesis

`synthesize.py` and `llm_client.py` form the generation layer. Gemini receives the structured finding and retrieved security evidence and produces a plain-language report. The intended role of the LLM is explanation and synthesis rather than independent vulnerability identification. This design supports audience-aware reporting without making the model the sole authority for CVE identity or deterministic severity.

The grounded prompt constrains generation to the supplied evidence. The system's anti-hallucination strategy is therefore layered: deterministic analysis controls risk properties, retrieval supplies evidence, citation validation checks references, and abstention provides an explicit failure path.

G. Citation Validation

Citation validation checks whether a generated CVE reference corresponds to an identifier available in the supported knowledge base and whether the cited evidence is appropriate to the finding. This mechanism is central to auditability. A report can contain fluent prose without being trustworthy; a validated citation provides a traceable link between a claim and an evidence record.

The grounded experiment provides a focused test of this behavior. Among 58 evaluated findings, 21 grounded cases issue citations and all 21 are correct. The corresponding ungrounded condition issues zero citations. These results are reported as citation issuance and citation correctness.

H. Abstention and UNSCORED Handling

The system can avoid synthesis when evidence is unavailable or insufficient. This behavior is important for security reporting because a missing answer is preferable to an unsupported vulnerability reference when the report is intended for audit or remediation. The evaluation further explores this principle using held-out CVEs and a similarity threshold, where the retriever can abstain when no candidate reaches the required similarity.

Retrieval abstention and production citation eligibility are evaluated as separate controls; the threshold study isolates the measurable behavior of lexical retrieval.

I. API, User Interface, and Auditability

The implementation exposes the processing workflow through a Flask API with token authentication and a Next.js user interface. the implementation also documents PDF/JSON export, an audit log, and a Playwright parity suite. These components support operational traceability and repeatable interaction with the assessment workflow.

Quota-aware degradation includes top-N synthesis behavior and a MockClient fallback. Such mechanisms preserve system operation under external API constraints, but they do not improve retrieval quality. They are therefore treated as deployment safeguards rather than evaluation variables.

V. METHODOLOGY AND IMPLEMENTATION

A. Software Components

TABLE II. SYSTEM COMPONENTS AND FUNCTIONS

Component	Function	Role
nmap_parser.py	Parse Nmap XML into structured observations	Input normalization
rule_engine.py	Apply deterministic security/risk rules	Risk analysis
retrieval.py	Rank CVE/CIS evidence using supported retrieval	Knowledge access
synthesize.py	Construct grounded narrative output	LLM synthesis
llm_client.py	Interface with Gemini / fallback client	Generation service
webapp.py	Expose API and application workflow	System integration

B. Technology Stack

The backend uses Python and Flask for the API layer. The user interface uses Next.js. Nmap supplies scan data, while CVE, CISA KEV, and CIS-oriented resources supply security knowledge. TF-IDF provides the documented lexical retrieval representation. Gemini supplies the language-generation component. The architecture is therefore a composition of deterministic software,

information retrieval, external security knowledge, and generative AI rather than a single trained model.

No model-training stage is present in the evaluated pipeline. The CVE benchmark is ingested as temporary retrieval knowledge and evaluated as a test corpus. The benchmark is used as a retrieval test corpus rather than a model-training corpus.

C. Knowledge Representation and Retrieval

For each knowledge entry, the textual fields are converted into TF-IDF features. The vector representation permits similarity-based ranking using the query and indexed entries. This approach is supported by standard TF-IDF implementations, where term frequency is reweighted by inverse document frequency and normalized vectors can be compared through cosine similarity [9].

The lexical design has two useful properties for this project: it is transparent and it avoids additional model training or inference cost during retrieval. Its main limitation is semantic sensitivity. A query using terminology different from the indexed description can receive a lower similarity score even when the concepts are related. The difference between truncated and paraphrased evaluation results provides direct evidence of this effect.

D. Grounding and Validation Logic

The synthesis layer receives retrieved context together with the finding. The report is expected to identify security evidence through CVE references and configuration guidance. Validation then checks the emitted citations. When evidence does not support a citation, the report should not silently substitute a guessed identifier. The documented design instead supports no-grounded-citation or UNSCORED behavior.

The sibling-CVE stress case described in the implementation, involving CVE-2017-0143 and CVE-2017-0144, is relevant because closely related identifiers can create a particularly difficult validation condition. The project identifies an unsupported_cve miss in the ungrounded condition as a live validation observation. This is treated as a qualitative stress-test result, not as a general benchmark statistic.

E. Operational Degradation

External LLM services can impose quota and availability constraints. The implementation therefore supports quota-aware degradation, including top-N synthesis and a MockClient fallback. These mechanisms allow the deterministic and retrieval layers to remain useful when full generation is unavailable. The fallback is treated as an operational safeguard and is excluded from retrieval and citation metrics.

F. Algorithmic Workflow

The documented system behavior can be expressed without introducing unsupported algorithmic detail as follows.

Algorithm 1: Grounded infrastructure-security reporting
Input: Nmap XML scan S
1: Parse S into structured observations.
2: Apply deterministic security rules to produce findings F.
3: For each finding $f \in F$, construct a retrieval query q.
4: Retrieve ranked CVE/CIS evidence E using the supported lexical retriever.
5: Construct a grounded context C from f and E.
6: If evidence is insufficient, mark the finding UNSCORED or manual-review-required.
7: Otherwise, generate a report narrative G using the grounded LLM.
8: Validate CVE citations and evidence references in G.
9: Accept the grounded report only when citation requirements are satisfied; otherwise emit the appropriate abstention state.
Output: Security report R or explicit abstention.

VI. EXPERIMENTAL SETUP AND EVALUATION
METHODOLOGY

A. Evaluation Objective

The evaluation is designed to answer a narrower question than vulnerability detection: whether the retrieval component can recover CVE knowledge and how confidence filtering changes the trade-off between precision and coverage. A separate small experiment examines citation behavior with and without grounded evidence. The evaluation does not train the cybersecurity system and does not call the LLM during retrieval scoring.

The benchmark is derived from the public CVE data associated with the Vul-RAG research. The evaluation corpus contains 1,325 unique CVEs represented by identifiers, weakness information, and descriptions. The code portion of the original source-code benchmark is ignored because the proposed system does not contain a source-code detection path.

B. Knowledge-Base Split

An 80/20 split produces 1,059 CVEs in a temporary retrieval knowledge base and 266 held-out CVEs. The held-out portion is used to examine whether the lexical retriever can recognize when an unseen CVE does not have an appropriate knowledge entry. This split is an evaluation construct and does not represent production training or deployment data.

TABLE III. EVALUATION DATASET AND QUERY CONSTRUCTION

Item	Value / description
Unique CVEs	1,325
Temporary knowledge base	1,059 CVEs (80%)
Held-out set	266 CVEs (20%)
Retrieval inference	No LLM calls

Query mode 1	Truncated query: first half of indexed description
Query mode 2	Paraphrased / knowledge query
Main retrieval measures	Top-1, Top-5, precision, coverage
Additional analysis	Similarity threshold and held-out abstention

C. Query Construction

Two query modes are evaluated. The truncated mode uses approximately half of the original indexed description and therefore measures partial-text matching under substantial lexical overlap. The paraphrased knowledge mode uses differently worded knowledge text and is treated as the more informative retrieval measure in the project analysis.

This distinction is important when interpreting the 93.6% Top-1 truncated result. the implementation explicitly cautions that the value is inflated by self-retrieval characteristics. The principal retrieval result is therefore reported using the paraphrased mode, where Top-1 is 51.6% and Top-5 is 72.4% for the joinable knowledge-query evaluation.

D. Metrics

$$\text{Top-k} = \frac{(\# \text{ queries with ground-truth CVE in top k})}{(\# \text{ evaluated queries})} \quad (2)$$

$$\text{Coverage} = \frac{(\# \text{ queries producing an accepted result})}{(\# \text{ evaluated queries})} \quad (3)$$

Citation precision is computed only over issued citations in the grounded-generation sample:

$$\text{Citation precision} = \frac{(\# \text{ correct issued citations})}{(\# \text{ issued citations})} \quad (4)$$

Unsupported citation rate is the complementary proportion among issued citations when the experiment provides that classification. Citation issuance itself is a count of cases in which a citation is produced and must not be interpreted as an accuracy measure.

E. Threshold Experiment

For truncated queries, threshold 0.4 yields approximately 95.1% coverage and 97.2% precision, with 1.2% of correct answers lost and 57.1% of unseen CVEs abstained. Because truncated queries share text with the indexed descriptions, these values should be interpreted as partial-text robustness rather than general retrieval accuracy.

The threshold sweep should be interpreted across its separate in-knowledge and held-out populations because they have different sizes and evaluation behavior. Knowledge-mode queries and unseen-CVE abstention operate on different query distributions and similarity scales.

F. Grounded Versus Ungrounded Citation Evaluation

A separate sample of 58 cases compares grounded and ungrounded generation behavior. The grounded condition issues 21 citations and all 21 are correct. The ungrounded condition issues zero citations. The sample is small and focuses

specifically on CVE citation issuance and correctness.

VII. RESULTS AND ANALYSIS

A. Retrieval Performance

TABLE IV. RETRIEVAL PERFORMANCE ACROSS QUERY MODES

Query mode	Top-1	Top-5	Interpretation
Truncated	93.6%	—	Partial-text self-retrieval; not the principal general retrieval accuracy
Paraphrased / knowledge	51.6%	72.4%	Different wording; primary retrieval result in the project analysis

Table IV shows a large difference between query modes. The 93.6% Top-1 result demonstrates that the lexical retriever can recover an entry when a substantial portion of the indexed description is reused; this result is specific to partial-text overlap. The paraphrased mode is more demanding and yields 51.6% Top-1 and 72.4% Top-5. The gap indicates sensitivity to lexical alignment, which is consistent with the TF-IDF design.

The Top-5 result is useful because a downstream report generator may benefit from a relevant candidate appearing within a short ranked list even when it is not ranked first. Top-5 retrieval recall and citation correctness remain separate evaluation measures.

B. Similarity Threshold Trade-Off

TABLE V. SIMILARITY THRESHOLD TRADE-OFF

Evaluation	Threshold	Precision	Coverage / abstention
Paraphrased knowledge	0.0	51.6%	Baseline coverage
Paraphrased knowledge	0.4	91.6%	≈ 18% coverage
Truncated	0.4	97.2%	95.1% coverage
Held-out CVEs	0.0	—	0/266 abstained
Held-out CVEs	0.4	—	57.1% abstained

Table V demonstrates the central retrieval trade-off. For paraphrased queries, increasing the threshold from 0.0 to 0.4 raises reported precision from 51.6% to 91.6% but leaves only about 18% of queries with accepted results. This behavior is consistent with a confidence filter: the system becomes more selective and therefore answers fewer queries.

The held-out results provide a second perspective. Without a threshold, none of the 266 unseen CVEs is rejected by the retriever. At threshold 0.4, 57.1% are abstained from. The experiment therefore identifies a concrete weakness in the baseline similarity search: without a confidence cutoff, lexical similarity always produces a candidate even when the ground-truth CVE is outside the knowledge base.

Threshold 0.4 is an evaluated operating point rather than a universal optimum. It is not yet

integrated into the production retrieval pipeline, and broader in-knowledge testing is needed to quantify the cost of discarding correct matches.

C. Grounded Versus Ungrounded Citation Behavior

TABLE VI. GROUNDED VERSUS UNGROUNDED CITATION BEHAVIOR

Condition	Cases	Citations issued	Correct citations	Citation precision
Grounded	58	21	21	100% of issued citations
Ungrounded	58	0	0	Not applicable

The grounded condition issues 21 citations among 58 cases, and every issued citation is correct. The ungrounded condition issues none. The immediate interpretation is that grounding plus validation constrains citation behavior in the evaluated sample. The result also exposes an important limitation: citation precision alone can be misleading when issuance is low. A system that refuses to cite anything can avoid citation errors while providing little evidence.

The result is best interpreted as a paired citation-behavior observation: grounded generation produces validated citations with 100% correctness among issued citations, while the ungrounded condition abstains from issuing citations in all 58 cases.

D. CIS Mapping Validation

Eight of eight evaluated CIS mappings matched the expected guidance. This supports the feasibility of mapping relevant findings to configuration guidance, but the sample is too small to characterize general CIS retrieval performance. The result is therefore kept separate from the CVE retrieval table and from citation precision.

E. Evaluation Pipeline

Figure 3 summarizes the evaluation procedure. The benchmark is split into knowledge and held-out subsets, query variants are constructed, lexical retrieval is evaluated, and threshold sweeps are applied. No LLM calls are required for the retrieval experiment, which isolates the retrieval behavior from generation variability.

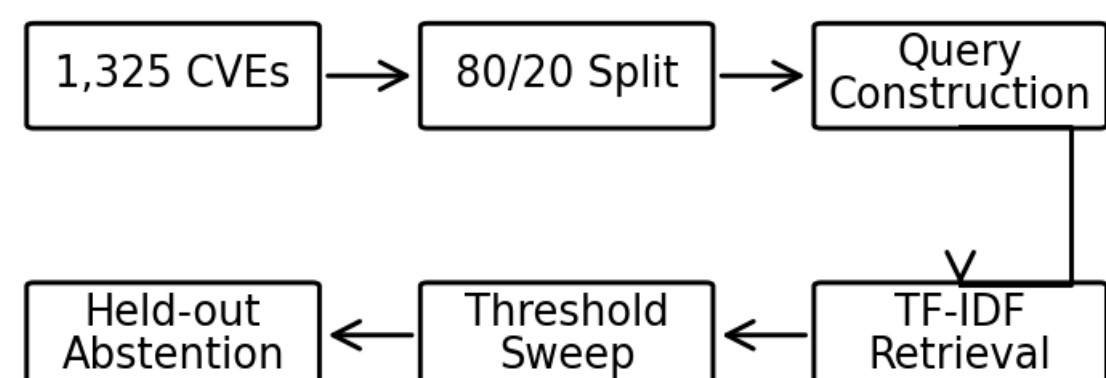


Fig. 3. Retrieval evaluation pipeline using the 1,325-CVE benchmark, query construction, Top-k scoring, threshold sweeps, and held-out abstention.

VIII. COMPARATIVE DISCUSSION

The architectural difference is also consequential.

Vul-RAG's knowledge-level representation is designed around vulnerability semantics and historical fixes, whereas the present system uses TF-IDF over supported CVE/CIS text. The proposed system uses a different retrieval abstraction. Similarly, the present synthesis stage is a single grounded generation pass rather than an iterative reasoning procedure matching the reference work.

The systems nevertheless share a common research principle: external security knowledge can constrain an LLM that otherwise operates from parametric knowledge. The present work extends that principle to a different application boundary, where the input is an infrastructure observation and the output is an auditable report.

IX. DISCUSSION

A. *Deterministic Reasoning Before Generation*

The deterministic-first design is a central architectural property because security severity and prioritization should remain stable when language-model generation changes. By calculating these properties before synthesis, the system constrains the LLM to explain a security state rather than independently inventing one. This separation also supports audibility: a reviewer can inspect the finding and the rule result without treating the generated prose as the source of truth.

B. *Grounding and Citation Auditability*

Grounding provides a mechanism for connecting generated statements to external evidence. In the evaluated sample, the grounded condition issues fewer than half of the possible case-level citations but achieves correctness for all issued citations. This suggests a conservative behavior in which evidence availability limits citation issuance. For security reporting, such conservatism can be useful, provided the interface makes abstention visible and does not present missing citations as complete coverage.

C. *Retrieval Limitations*

The 51.6% Top-1 and 72.4% Top-5 paraphrased results show that lexical retrieval does not consistently recover the exact ground-truth CVE when query wording differs from indexed text. The threshold experiment improves precision but sharply reduces coverage. This is a classic retrieval operating-point trade-off, and it indicates that confidence filtering alone cannot resolve semantic mismatch.

D. *Practical Implications*

The framework is most naturally suited to settings where scan observations need to be

transformed into reviewable security summaries. Its deterministic rules, evidence records, citation validation, and audit logging provide multiple points at which a reviewer can inspect or reject an automated conclusion. The approach is intended to support expert validation for ambiguous findings by providing a structured path from machine-readable observations to evidence-linked reporting.

E. *Generalization*

The retrieval evaluation is performed on Linux-kernel CVE descriptions derived from the Vul-RAG benchmark, whereas production input is Nmap network data. Broader infrastructure benchmarks are required to characterize deployment-domain generalization.

X. THREATS TO VALIDITY AND LIMITATIONS

- Domain mismatch: the retrieval benchmark consists of Linux-kernel CVEs, while production inputs are Nmap scan findings.
- Query-construction bias: truncated queries share text with indexed descriptions and therefore produce an easier retrieval task than paraphrased queries.
- Lexical retrieval limitation: TF-IDF depends on vocabulary overlap and may miss semantically related descriptions expressed with different terminology.
- Threshold evaluation scope: the threshold is measured experimentally but is not yet integrated into the production retrieval pipeline.
- Coverage trade-off: the 0.4 threshold increases reported precision for paraphrased queries while reducing coverage to approximately 18%; therefore, it cannot be treated as an unconditional improvement.
- Held-out evaluation interpretation: the 266 held-out CVEs and the 1,059 in-knowledge entries represent different populations, so their coverage and abstention results should not be merged into one metric.
- Grounded-generation sample size: the 58-case citation experiment is small and should not be generalized to all security reports or all LLM outputs.
- Production safeguard separation: the retrieval benchmark does not test the production rule that only CVE IDs present in the knowledge base can be cited.
- No source-code detection: the framework does not implement the source-code vulnerability-classification task evaluated by Vul-RAG.

- External LLM constraints: Gemini API availability and quota can affect generation behavior and require fallback or degraded modes.
- CIS validation scale: the reported 8/8 CIS mapping result is a small validation sample rather than a comprehensive benchmark.

XI. FUTURE WORK

A higher-value validation experiment should evaluate the complete pipeline on a small set of Nmap scans whose exposed services and versions have independently verified CVE associations. The proposed method should be compared with a CPE-matching baseline using identical scan observations. The evaluation should report CVE identification precision and recall, service-to-CVE matching coverage, citation correctness, abstention rate, and end-to-end report validity. The baseline would map observed service/version information to CPE identifiers and match those CPEs against NVD vulnerability records. No independently verified Nmap scan set with known CVE labels is included in the current evidence, so this experiment is specified as future validation rather than reported as a completed result.

The most direct future direction is integration of similarity-confidence thresholds into the production retrieval pipeline. The evaluation demonstrates that a threshold can materially change the precision-coverage operating point and can enable abstention on held-out CVEs. Production integration should therefore be accompanied by a larger in-knowledge evaluation so that both false citations and discarded correct answers are measured.

The evaluation should also expand beyond Linux-kernel CVEs to vulnerability descriptions and service ecosystems more representative of Nmap findings. Additional security knowledge sources can be incorporated, including broader configuration guidance and exploitation-priority information, provided that provenance and citation validation remain explicit.

Larger grounded-generation experiments should examine citation issuance, citation correctness, unsupported claims, and abstention together. Evaluation should distinguish the ability to retrieve evidence from the ability of the LLM to use that evidence faithfully. A broader benchmark could also include manually verified security findings and expert assessment of report usefulness, but such a study is not part of the current evidence.

Finally, auditability can be strengthened by recording the finding, retrieval query, retrieved evidence, similarity scores, generation state,

citation-validation decision, and final report state as a reproducible chain. This would support later review without conflating operational logs with model-quality metrics.

XII. CONCLUSION

This paper presents a grounded RAG framework for infrastructure-security assessment and auditable report generation from Nmap scan information. The architecture separates deterministic security analysis from evidence retrieval and language-model synthesis, while citation validation and explicit abstention states provide controls against unsupported vulnerability references. CVE-oriented knowledge and CIS configuration guidance support both vulnerability context and remediation-oriented reporting.

These findings support the use of retrieval and grounding as measurable components of trustworthy security reporting, while also exposing the limitations of lexical matching and small-sample citation evaluation. The contribution is an auditable infrastructure-security application of grounded RAG. Confidence-aware retrieval, broader infrastructure benchmarks, and larger grounded-generation evaluations remain necessary to establish wider generalization.

REFERENCES

- [1] G. Lyon, "Nmap XML output," Nmap Network Scanning, Nmap Project. [Online]. Available: <https://nmap.org/book/output-formats-xml-output.html>. Accessed: Sep. 28, 2026.
- [2] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Trans. Inf. Syst.*, vol. 43, no. 2, 2025, Art. no. 35, doi: 10.1145/3703155.
- [3] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, 2023, Art. no. 248, doi: 10.1145/3571730.
- [4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020.
- [5] X. Du, G. Zheng, K. Wang, Y. Zou, Y. Wang, W. Deng, J. Feng, M. Liu, B. Chen, X. Peng, T. Ma, and Y. Lou, "Vul-RAG: Enhancing LLM-based vulnerability detection via knowledge-level RAG," *ACM Trans. Softw. Eng. Methodol.*, vol. 35, no. 10, 2026, Art. no. 312, doi: 10.1145/3797277.
- [6] National Institute of Standards and Technology, "National Vulnerability Database," NIST. [Online]. Available: <https://www.nist.gov/itl/nvd>. Accessed: Sep. 28, 2026.
- [7] Cybersecurity and Infrastructure Security Agency, "Known Exploited Vulnerabilities Catalog," CISA. [Online]. Available: <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>. Accessed: Sep. 28, 2026.
- [8] Center for Internet Security, "CIS Benchmarks," CIS. [Online]. Available: <https://www.cisecurity.org/cis-benchmarks-overview>. Accessed: Sep. 28, 2026.

- [9] scikit-learn developers, “TfidfVectorizer,” scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html. Accessed: Sep. 28, 2026.
- [10] P. Bécharde and O. Marquez Ayala, “Reducing hallucination in structured outputs via Retrieval-Augmented Generation,” in Proc. NAACL-HLT: Industry Track, 2024, pp. 191 – 200, doi: 10.18653/v1/2024.naacl-industry.19.